Bayesian statistics with R
6. Bayesian analyses in R with Jags (and brms)

Olivier Gimenez

November-December 2026

Bayes in practice

Software implementation (R friendly)

Oldies but goodies:

- WinBUGS, OpenBUGS: Where it all began.
- Jags: What we will use in this course.

Software implementation (R friendly)

Oldies but goodies:

- WinBUGS, OpenBUGS: Where it all began.
- Jags: What we will use in this course.

The new kids on the block:

- Nimble: What I'm going for these days.
- Stan: Entirely different algorithmic approach.
- Greta: Dunno anything about it.

Software implementation (R friendly)

Oldies but goodies:

- WinBUGS, OpenBUGS: Where it all began.
- Jags: What we will use in this course.

The new kids on the block:

- Nimble: What I'm going for these days.
- Stan: Entirely different algorithmic approach.
- Greta: Dunno anything about it.

If you're not into coding:

- brms: Bayesian regression models with Stan.
- MCMCglmm: Generalised Linear Mixed Models.
- Check out the CRAN Task View: Bayesian Inference for more.

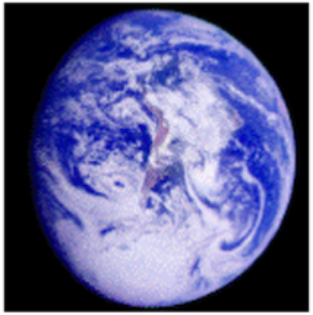
Introduction to JAGS (Just Another Gibbs Sampler)

Martyn Plummer



Real example

Impact of climatic conditions on white stork breeding success





Let's do a logistic regression on some White stork data

- Assess effects of temperature and rainfall on productivity.

Let's do a logistic regression on some White stork data

- Assess effects of temperature and rainfall on productivity.
- We have collected data.

Let's do a logistic regression on some White stork data

- Assess effects of temperature and rainfall on productivity.
- We have collected data.
- We need to build a model - write down the likelihood.

Let's do a logistic regression on some White stork data

- Assess effects of temperature and rainfall on productivity.
- We have collected data.
- We need to build a model - write down the likelihood.
- We need to specify priors for parameters.

Read in the data

```
nbchicks <- c(151,105,73,107,113,87,77,108,118,122,112,120,122,89,69,71,
              53,41,53,31,35,14,18)

nbpairs <- c(173,164,103,113,122,112,98,121,132,136,133,137,145,117,
             90,80,67,54,58,39,42,23,23)

temp <- c(15.1,13.3,15.3,13.3,14.6,15.6,13.1,13.1,15.0,11.7,15.3,14.4,
          14.4,12.7,11.7,11.9,15.9,13.4,14.0,13.9,12.9,15.1,13.0)

rain <- c(67,52,88,61,32,36,72,43,92,32,86,28,57,55,66,26,28,96,48,90,86,
          78,87)
```

```
datax <- list(N = 23,
              nbchicks = nbchicks,
              nbpairs = nbpairs,
              temp = (temp - mean(temp))/sd(temp),
              rain = (rain - mean(rain))/sd(rain))
```

Write down the model

$$\begin{aligned}
\text{nbchicks}_i &\sim \text{Binomial}(\text{nbpairs}_i, p_i) && \text{[likelihood]} \\
\text{logit}(p_i) &= a + b_{temp}\ \text{temp}_i + b_{rain}\ \text{rain}_i && \text{[linear model]} \\
a &\sim \text{Normal}(0, 1.5) && \text{[prior for } a\text{]} \\
b_{temp} &\sim \text{Normal}(0, 1.5) && \text{[prior for } b_{temp}\text{]} \\
b_{rain} &\sim \text{Normal}(0, 1.5) && \text{[prior for } b_{rain}\text{]}
\end{aligned}$$

Build the model

```
{
# Likelihood
    for( i in 1 : N){
        nbchicks[i] ~ dbin(p[i],nbpairs[i])
        logit(p[i]) <- a + b.temp * temp[i] + b.rain * rain[i]
        }
# ...
```

Specify priors

```
# Priors
a ~ dnorm(0,1/(1.5^2))
b.temp ~ dnorm(0,1/(1.5^2))
b.rain ~ dnorm(0,1/(1.5^2))
}
```

Warning: Jags uses precision for Normal distributions (1 / variance), not standard deviation.

You need to write everything in a file

```
model <-
paste("
model
{
    for( i in 1 : N)
        {
        nbchicks[i] ~ dbin(p[i],nbpairs[i])
        logit(p[i]) <- a + b.temp * temp[i] + b.rain * rain[i]
        }
a ~ dnorm(0,1/(1.5^2))
b.temp ~ dnorm(0,1/(1.5^2))
b.rain ~ dnorm(0,1/(1.5^2))
    }
")
```

Alternatively, you may write a R function

```
logistic <- function() {
    for( i in 1 : N)
        {
        nbchicks[i] ~ dbin(p[i],nbpairs[i])
        logit(p[i]) <- a + b.temp * temp[i] + b.rain * rain[i]
        }

# priors for regression parameters
a ~ dnorm(0,1/(1.5^2))
b.temp ~ dnorm(0,1/(1.5^2))
b.rain ~ dnorm(0,1/(1.5^2))
    }
```

Let us specify a few additional things

```
# list of lists of initial values (one for each MCMC chain)
init1 <- list(a = -0.5, b.temp = -0.5, b.rain = -0.5)
init2 <- list(a = 0.5, b.temp = 0.5, b.rain = 0.5)
inits <- list(init1,init2)

# specify parameters that need to be estimated
parameters <- c("a","b.temp","b.rain")

# specify nb iterations for burn-in and final inference
nb.burnin <- 10000
nb.iterations <- 20000 # beware: nb.iterations includes nb.burnin!
```

Run Jags

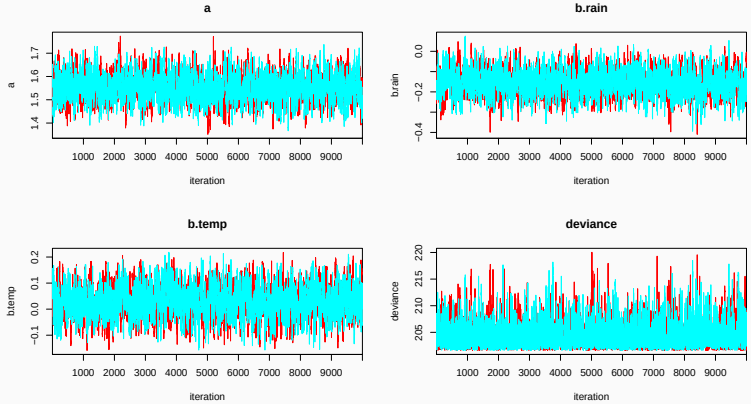
```
# load R2jags
library(R2jags)
# run Jags
storks <- jags(data  = datax,
               inits = inits,
               parameters.to.save = parameters,
               #model.file = "code/logistic.txt",
               model.file = logistic, # if a function was written
               n.chains = 2,
               n.iter = nb.iterations,
               n.burnin = nb.burnin)
storks
```

Inspect parameter estimates

```
#> Compiling model graph
#>    Resolving undeclared variables
#>    Allocating nodes
#> Graph information:
#>    Observed stochastic nodes: 23
#>    Unobserved stochastic nodes: 3
#>    Total graph size: 185
#>
#> Initializing model
#> Inference for Bugs model at "code/logistic.txt", fit using jags,
#>  2 chains, each with 20000 iterations (first 10000 discarded)
#>  n.sims = 20000 iterations saved. Running time = 0.336 secs
#>          mu.vect sd.vect    2.5%     25%     50%     75%   97.5%  Rhat n.eff
#> a         1.553   0.055   1.446   1.515   1.552   1.589   1.663 1.002   980
#> b.rain   -0.154   0.060  -0.272  -0.196  -0.155  -0.113  -0.037 1.001  8500
#> b.temp    0.030   0.057  -0.082  -0.009   0.030   0.069   0.142 1.001 20000
#> deviance 204.462   2.298 201.787 202.761 203.906 205.552 210.380 1.002  2600
```

Trace plot

```
traceplot(storks, ask = FALSE, mfrow = c(2, 2))
```



Your turn: Practical 5

MCMC makes you queens and kings of the stats world

Get all values sampled from posteriors

```
res <- as.mcmc(storks) # convert outputs in a list
res <- rbind(res[[1]],res[[2]]) # put two MCMC lists on top of each other
head(res)
#>              a     b.rain     b.temp deviance
#> [1,] 1.473195 -0.2571849 0.08437669 206.2077
#> [2,] 1.448817 -0.2403418 0.08294533 206.8596
#> [3,] 1.457597 -0.2118908 0.07165321 205.2889
#> [4,] 1.431497 -0.1664404 0.03945974 206.3715
#> [5,] 1.452194 -0.1525438 0.02540322 204.9266
#> [6,] 1.481456 -0.2019104 0.02264394 203.7732
```

```
tail(res)
#>                  a      b.rain      b.temp deviance
#> [19995,] 1.477898 -0.14664659  0.08061350 204.2299
#> [19996,] 1.541782 -0.15038229  0.09818288 203.1093
#> [19997,] 1.548562 -0.09100646  0.02538905 202.7532
#> [19998,] 1.551563 -0.09943886  0.02532323 202.4544
#> [19999,] 1.553647 -0.08503655 -0.06185470 204.7226
#> [20000,] 1.553961 -0.09599017 -0.05392312 204.0833
```

Compute a posteriori Pr(rain < 0)

```
# probability that the effect of rainfall is negative
mean(res[,'b.rain'] < 0)
#> [1] 0.99555
```

Compute a posteriori Pr(temp < 0)

```
# probability that the effect of temperature is negative
mean(res[,'b.temp'] < 0)
#> [1] 0.30205
```

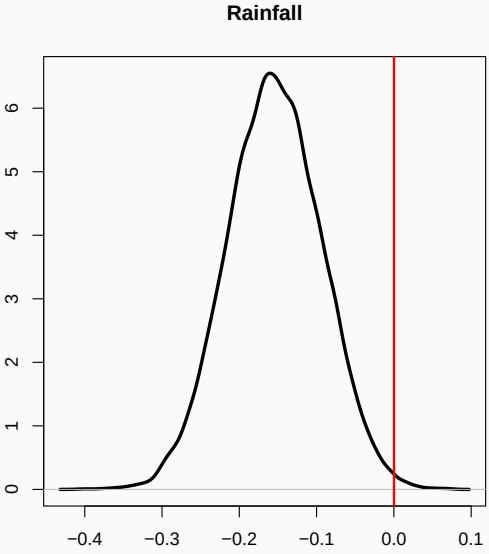
Get credible interval for the rain effect

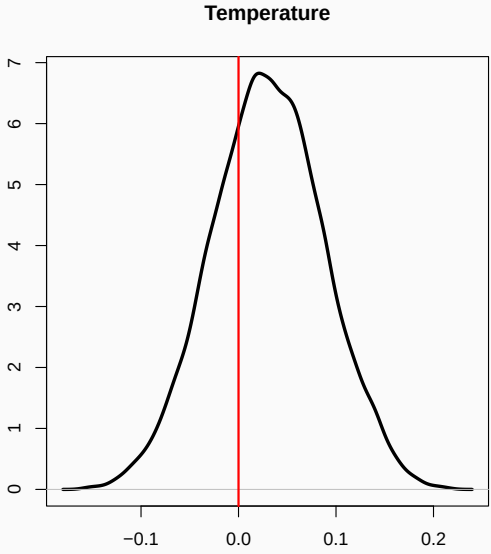
```
quantile(res[,'b.rain'],c(0.025,0.975))
#>        2.5%       97.5%
#> -0.27202156 -0.03719511
```

Get credible interval for the temperature effect

```
quantile(res[,'b.temp'],c(0.025,0.975))
#>         2.5%        97.5%
#> -0.08193902  0.14180880
```

Graphical summaries





Your turn: Practical 6

Quick intro to `brms`

What is `brms`?

- Developed by Paul-Christian Bürkner.

What is `brms`?

- Developed by Paul-Christian Bürkner.
- `brms` fits Bayesian regression models, including GLMMs, using a syntax similar to `lme4`.

What is `brms`?

- Developed by Paul-Christian Bürkner.
- `brms` fits Bayesian regression models, including GLMMs, using a syntax similar to `lme4`.
- It uses `Stan` behind the scenes: no need to write Stan code yourself.

What is `brms`?

- Developed by Paul-Christian Bürkner.
- `brms` fits Bayesian regression models, including GLMMs, using a syntax similar to `lme4`.
- It uses `Stan` behind the scenes: no need to write Stan code yourself.
- See the vignettes and blog posts.

What is `brms`?

- Developed by Paul-Christian Bürkner.
- `brms` fits Bayesian regression models, including GLMMs, using a syntax similar to `lme4`.
- It uses `Stan` behind the scenes: no need to write Stan code yourself.
- See the vignettes and blog posts.
- A convenient alternative to Jags for models supported by `brms`.

Example: survival over one winter

Say we capture, mark and release $n = 57$ animals at the beginning of a winter, out of which we observe $y = 19$ animals alive at the end of the winter.

```
dat <- data.frame(alive = 19, # number of successes
                  total = 57) # number of trials
dat
#>   alive total
#> 1    19    57
```

Write down the model

Assuming independent animals, a common survival probability θ:

$$y \sim \text{Binomial}(n, \theta) \qquad \text{[likelihood]}$$
$$\theta \sim \text{Uniform}(0, 1) \qquad \text{[prior]}$$

We would like to estimate winter survival θ.

A simple estimate is $y/n = 19/57 \approx 0.333$.

Maximum-likelihood estimation

Fit an intercept-only logistic regression with grouped binomial data:

```
mle <- glm(cbind(alive, total - alive) ~ 1,
           family = binomial("logit"),
           data = dat)
```

Maximum-likelihood estimation

On the logit scale:

```
coef(mle)
#> (Intercept)
#>  -0.6931472
```

Back-transform to the probability scale:

```
plogis(coef(mle))
#> (Intercept)
#>   0.3333333
```

Bayesian analysis with brms

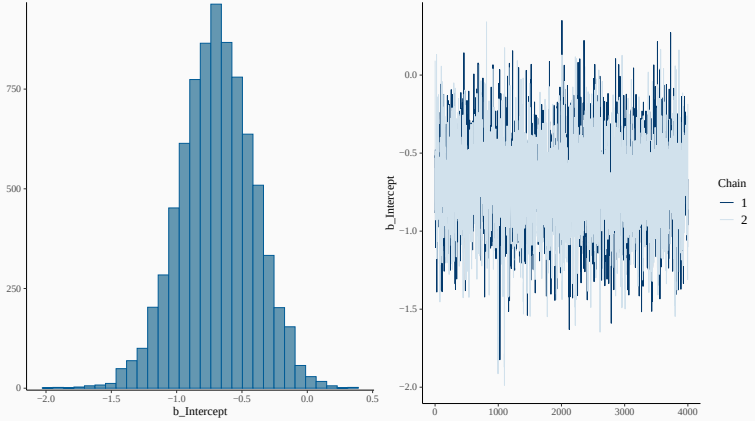
```
library(brms)
bayes.brms <- brm(alive | trials(total) ~ 1,
                  family = binomial("logit"),
                  data = dat,
                  chains = 2,
                  iter = 5000,    # iterations per chain, including warmup
                  warmup = 1000, # warmup (burn-in)
                  thin = 1)
```

Display results

```
bayes.brms
#>  Family: binomial
#>   Links: mu = logit
#> Formula: alive | trials(total) ~ 1
#>    Data: dat (Number of observations: 1)
#>   Draws: 2 chains, each with iter = 5000; warmup = 1000; thin = 1;
#>          total post-warmup draws = 8000
#>
#> Regression Coefficients:
#>           Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
#> Intercept    -0.69      0.28    -1.25    -0.15 1.00     2674     3182
#>
#> Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
#> and Tail_ESS are effective sample size measures, and Rhat is the potential
#> scale reduction factor on split chains (at convergence, Rhat = 1).
```

Visualize results

```
plot(bayes.brms)
```



Working with MCMC draws

Extract the posterior draws on the **logit scale**:

```
library(posterior)
draws_fit <- as_draws_matrix(bayes.brms)
knitr::kable(head(draws_fit[, "b_Intercept", drop = FALSE], 10))
```

b_Intercept
-0.8837582
-0.8697904
-0.5152412
-0.5303074
-0.4788605
-0.9264644
-1.1039624
-0.5572751
-0.6209021
-0.9808052

Working with MCMC draws

Back-transform each draw and summarize the posterior distribution of survival:

```
survival_draws <- plogis(draws_fit[, "b_Intercept"])
summarize_draws(survival_draws)
#> # A tibble: 1 x 10
#>   variable     mean median     sd    mad    q5   q95  rhat ess_bulk ess_tail
#>   <chr>       <dbl>  <dbl>  <dbl>  <dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
#> 1 b_Intercept 0.336  0.334 0.0618 0.0617 0.238 0.442  1.00    2674.    3182.
```

Working with MCMC draws

```
# Posterior probability that survival is below 0.5
mean(survival_draws < 0.5)
#> [1] 0.993625

# 95% credible interval for survival
quantile(survival_draws, c(0.025, 0.975))
#>      2.5%     97.5%
#> 0.2225933 0.4631649
```

What prior does `brms` use by default?

```
prior_summary(bayes.brms)
#> Intercept ~ student_t(3, 0, 2.5)
```

What if I want a different prior instead?

Specify a Normal prior with standard deviation 1.5 on the logit-scale intercept:

```
nlprior <- prior(normal(0, 1.5), class = "Intercept")
bayes.brms.prior <- brm(alive | trials(total) ~ 1,
                        family = binomial("logit"),
                        data = dat,
                        prior = nlprior,
                        chains = 2,
                        iter = 5000,
                        warmup = 1000,
                        thin = 1)
```

Priors

Double-check the prior that was used:

```
prior_summary(bayes.brms.prior)
#> Intercept ~ normal(0, 1.5)
```